

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80
81     } else {
82     //it scrolled out of view
83     t.appeared = false;
84
85     }
86
87 };
88
89 //create a modified fn with some additional logic
90 var modifiedFn = function() {
91
92     //mark the element as visible
93     t.appeared = true;
94
95     //is this supposed to happen only once?
96     if (settings.one) {
97
98         //remove the check
99         w.unbind('scroll', check);
100         var i = $.inArray(check, $.fn.appear.checks);
101         if (i >= 0) $.fn.appear.checks.splice(i, 1);
102
103     }
104
105     //trigger the original fn
106     fn.apply(this, arguments);
107
108 };
109
110 //bind the modified fn to the element
111 $(settings.one) t.one('appear', settings.data, modifiedFn);
112 settings.data, modifiedFn);

```



MATERIA:

Programación Orientada a Objetos

Unidad 1: Programación y Principios de Diseño Orientado a Objetos.

Docente:

Grupo:

Unidad #1 - Programación y Principios de Diseño Orientado a Objetos.

1.8. Constructores.

1.9. Métodos get y set.

1.8. Principios de diseño

- 1.8.1. Introducción y aplicación de los principios SOLID.
- 1.8.3. Aplicación básica de SOLID.
- 1.8.2. Buenas prácticas de diseño de clases y métodos.
- 1.8.4. Cohesión, acoplamiento, SRP y modularidad.
- 1.8.5. Patrones básicos: Singleton, DTO, MVC.

1.8. El constructor-El “encendido” de un objeto.

Es un método especial que se invoca automáticamente al crear un objeto con la palabra clave `new`.

Objetivo principal: Inicializar los atributos del objeto para que nazca en un estado válido y consistente.

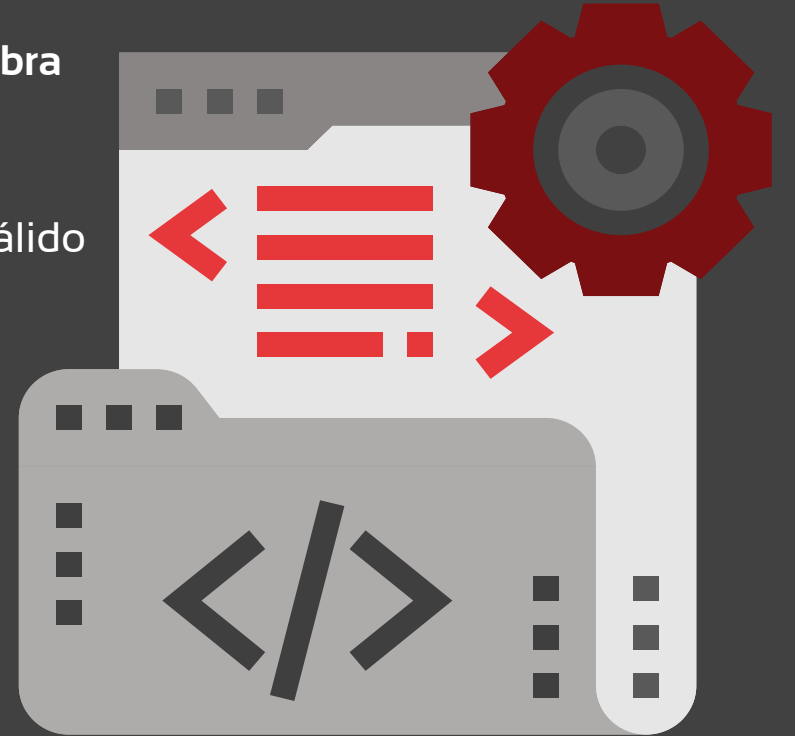
Analogía: Es como el manual de instrucciones para ensamblar un producto nuevo; garantiza que todo esté en su lugar desde el principio.

Características Clave:

- Se llama exactamente igual que la clase.

No tiene tipo de retorno, ni siquiera `void`.

Su trabajo es configurar, no devolver un valor.



1.9. Métodos get y set - POJOS

Encapsulación - Protegiendo los Datos

Getters y Setters: Guardianes de los Atributos

Encapsulación: Pilar de la POO que consiste en ocultar el estado interno de un objeto (atributos **private**).

¿Por qué? Para proteger los datos de modificaciones indebidas y controlar el acceso.

Getters (**get**): Métodos públicos para leer el valor de un atributo.

- **public String getMarca()**

Setters (**set**): Métodos públicos para modificar el valor de un atributo.

- Permiten añadir lógica de validación.
- **public void setAnio(int nuevoAnio)**

1.8. Principios de diseño, más allá del código que funciona

Conceptos Clave del Buen Diseño

- **Alta Cohesión:** Una clase tiene alta cohesión cuando todos sus elementos (métodos y atributos) están estrechamente relacionados y trabajan juntos para cumplir una única y bien definida tarea.
 - *Ejemplo:* Una clase **CalculadorDeImpuestos** solo calcula impuestos.
- **Bajo Acoplamiento:** Los cambios en una clase no deben forzar cambios en otras.
 - *Analogía:* Un sistema de LEGOs (bajo acoplamiento) vs. un castillo de naipes (alto acoplamiento).
- **Modularidad:** Es el resultado de lo anterior. El sistema se divide en módulos independientes y reemplazables.
- **SRP (Principio de Responsabilidad Única):** La materialización de la alta cohesión.

Principios SOLID - La Guía del Buen Programador

S.O.L.I.D. es un acrónimo de 5 principios para crear software mantenible y flexible.

S - Single Responsibility Principle: Una clase, una sola razón para cambiar.

O - Open/Closed Principle: Abierto a extensión, cerrado a modificación.

L - Liskov Substitution Principle: Los subtipos deben ser sustituibles por sus tipos base.

I - Interface Segregation Principle: Interfaces pequeñas y específicas en lugar de una grande.

D - Dependency Inversion Principle: Dependier de abstracciones, no de implementaciones concretas.

Patrones Básicos: Singleton y DTO

Patrones de Diseño - Soluciones Probadas

Singleton:

Propósito: Garantizar una única instancia de una clase.

Uso Típico: Conexiones a bases de datos, gestores de configuración.

Implementación: Constructor privado y un método estático getInstance().

DTO (Data Transfer Object):

Propósito: Un "mensajero" simple para transportar datos entre capas.

Características: No tiene lógica de negocio, solo atributos y sus getters/setters.

Uso Típico: Transferir datos de una API al controlador.

Patrón de Arquitectura MVC

MVC: Model-View-Controller

Es un patrón de arquitectura que separa la aplicación en tres componentes interconectados para organizar el código y reducir el acoplamiento.

Modelo (Model):  El "cerebro".

Vista (View):  La "cara".

Controlador (Controller):  El "intermediario".